TP 9 PowerShell Initiation

Durée approximative de cet atelier : 1 heure.

Objectifs

- Approfondir ses connaissances dans la pratique du langage PowerShell. Trois annexes d'aide sont disponibles à la dernière page de ce document.

Mise en place de l'atelier

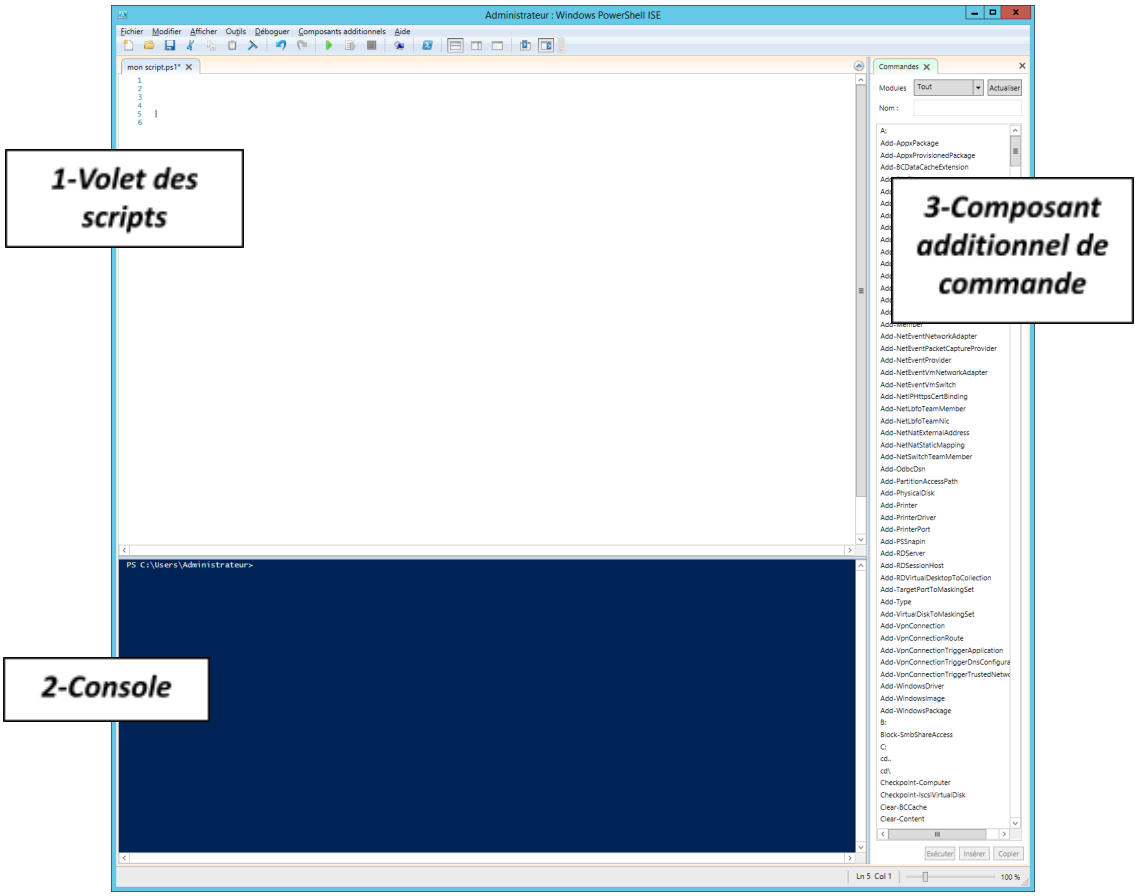
Pour réaliser cet exercice, vous devez disposer :

- ***D'une machine Windows 2016 serveur*** (avec un compte administrateur local)
- ***Disposer d'une partition E:\ en NTFS.***

Recherchez le programme " **Windows PowerShell ISE** " s'il n'est pas présent sur la barre des tâches.

Exécutez ***Windows PowerShell ISE*** ***en tant qu'administrateur***.

Paramétrez l'affichage d'ISE pour avoir l'affichage suivant :



(1) : Un éditeur de texte pour créer ses propres fichiers de scripts (extension "ps1")
(2) : La zone d'affichage du résultat de l'exécution des scripts et commandes
(3) : Le volet de commandes pour afficher l'aide et les paramètres liés à la commande

Dans la console Windows PowerShell ISE cliquez sur le menu Fichier Enregistrer sous…

Sélectionnez le dossier "**E:\TP_Powershell**" (créez le dossier **TP_Powershell** s'il n'existe pas) et donnez un nom à votre fichier "**.PS1**" (ex: TP_PS.ps1), cliquez sur Enregistrer.

Pensez dès à présent à **enregistrer souvent votre travail** dans la console ***Windows PowerShell ISE*** en cliquant sur l'icône :

PRÉPARATION

Mise à jour de l'aide de PowerShell :

Sur votre machine physique copiez le dossier "v5.1" qui contient l'aide de PowerShell à cet emplacement :
"S:\COURS_WINDOWS_Ateliers\Outils\PowerShell\Powershell initiation"
Collez le dossier "V5.1" dans le C:\ de votre machine virtuelle

Exécutez la commande : **Update-Help -UICulture en-us -SourcePath "C:\v5.1" -Force**

L'aide est mise à jour. (Un message d'erreur en rouge peut apparaître, ne pas en tenir compte…)

EXERCICE 9.1 : DÉCOUVERTE DU LANGAGE

Le but de cet exercice est de découvrir les premières commandes PowerShell.

1/ A l'aide de la commande : **Get-Variable** trouvez la version de la console PowerShell que vous utilisez

Commande: ……

$host ou get-host
Version4

2/ Faites-en sorte d'afficher uniquement la Version de la console PowerShell :

Notez ici votre commande :

$host.Version

3/ Que retourne la commande : **Get-Help *** ?

Alias fonctions et Cmdlets

4/ Que retourne la commande : **Get-Command** ?

Les CMDLETS, les fonctions, les alias

5/ En vous aidant de l'aide (avec -examples) lié à la commande **Get-command**, trouvez la syntaxe pour que :

Get-Command ne retourne que des CMDLETS
Get-Command ne retourne que des CMDLETS dont le nom contient "Item". Pensez au caractère générique: ***

Get-Help get-command -full pour trouver le paramètre -CommandType
Get-Command -CommandType cmdlet
get-command -CommandType cmdlet -Name *item*

6/ Saisissez **dir** puis **Get-Command dir**

Quel est le type de cette commande ? **Alias sur Get-ChildItem**……..

En exécutant **Get-ChildItem**, quelles différences faites-vous par rapport à la commande **dir** ?

Aucune puisqu'on utilise la cmdlet plutôt que l'alias

7/ Exécutez la commande : **Get-ChildItem –path "C:\" | gm**
Que voyez-vous? (Analysez les différents : "TypeName")

Les propriétés et les méthodes de l'objet liés aux :

fichiers (TypeName : System.IO.FileInfo)
et aux dossiers (TypeName : System.IO.DirectoryInfo)

8/ Exécutez les commandes : **Get-Help New-Item –Full** et **Get-help Add-Content -Full**
Lisez bien les exemples et créez un fichier "**test.txt**" dans le dossier "**E:**" en ajoutant 2 lignes avec le mot "**AZERTY**"

Notez les syntaxes utilisées :

1ère méthode:

```
New-item -Itemtype File -path "E:\test.txt"
Add-Content -Path "E:\test.txt" -Value "AZERTY"
Add-Content -Path "E:\test.txt" -Value "AZERTY"
```

2ème méthode:

```
Add-Content -Path "E:\test.txt" -Value "AZERTY"
Add-Content -Path "E:\test.txt" -Value "AZERTY"
```

3ème méthode

```
New-item -Itemtype File -path "E:\test.txt" -Value "AZERTY`r`nAZERTY"
```

4ème méthode

```
Add-Content -Path "E:\test.txt" -Value "AZERTY`r`nAZERTY"
```

5ème méthode

```
Write-output "AZERTY" | out-file -FilePath "E:\test.txt" -Append
Write-output "AZERTY" | out-file -FilePath "E:\test.txt" -Append
```

9/ Exécutez la commande: **Get-item –path "E:\test.txt" | gm**

Trouvez les commandes pour:

Afficher la taille en Octet du fichier (Get-item –path "E:\test.txt").Length

Afficher la date de création du fichier (Get-item –path "E:\test.txt").CreationTime

Afficher la date du dernier accès du fichier (Get-item –path "E:\test.txt").LastAccessTime

Afficher le chemin complet du fichier (Get-item –path "E:\test.txt").FullName

Afficher le nom du fichier (Get-item –path "E:\test.txt").Name

Afficher le nom du dossier ou lecteur d'appartenance (Get-item –path "E:\test.txt").DirectoryName

Trouvez une méthode (méthode() vu en cours) pour supprimer le fichier : **"E:\test.txt" :**

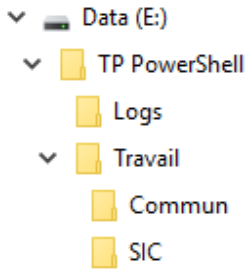
```
(Get-item -path "E:\test.txt").delete()
```

STOP
PASSER PPTX
Commande de bases

Exercice 9.2 : Commande de bases sur les dossiers et fichiers

Le but de cet exercice est de manipuler des fichiers et des répertoires en PowerShell.

1/ En vous appuyant sur l'aide en ligne de la commande " **New-Item** ", Créez les dossiers suivants en 5 lignes de commandes:



```
New-Item -ItemType Directory -Name "TP Powershell" -path "e:\"
New-Item -ItemType Directory -Name "Travail" -path "e:\TP Powershell"
New-Item -ItemType Directory -Name "Logs" -path "e:\TP Powershell"
New-Item -ItemType Directory -Name "SIC" -path "e:\TP Powershell\Travail"
New-Item -ItemType Directory -Name "Commun" -path "e:\TP Powershell\Travail"
```

Pouvait-on faire plus simple? (en 3 lignes de commandes)

Oui avec mkdir:

```
mkdir "e:\TP PowerShell\Travail\SIC"
mkdir "e:\TP PowerShell\Travail\Commun"
mkdir "e:\TP PowerShell\Logs"
```

ou avec new-item

```
New-item -Itemtype Directory -path "E:\TP PowerShell\Travail\SIC"
New-item -Itemtype Directory -path "E:\TP PowerShell\Travail\Commun"
New-item -Itemtype Directory -path "E:\TP PowerShell\Logs"
```

2/ Créez dans " **Travail** " un fichier nommé " **Test.txt** " contenant la ligne " **Fichier test créé en PowerShell** ".
Créez aussi un fichier "**powershell.txt"** dans le dossier : "**TP PowerShell"**

Notez les syntaxes utilisées :

```
New-Item -Type file -Name "test.txt" -Value "Fichier test crée en PowerShell" -path "e:\TP Powershell\Travail"
Ou
New-Item -Type file -path "e:\TP Powershell\Travail\test.txt" -Value "Fichier test crée en PowerShell"
Ou
add-content -Value "Fichier test crée en PowerShell" -path "e:\TP Powershell\Travail\test.txt"
```

```
New-Item -Type file -Name "PowerShell.txt" -path "e:\TP Powershell"
Ou
New-Item -Type file -path "e:\TP Powershell\ PowerShell.txt"
ou
add-content -value "" -path "e:\TP Powershell\ PowerShell.txt "
```

3/ Exécutez la commande suivante : **Get-ChildItem -path "E:\TP Powershell"**

Qu'affiche cette commande ?

Les fichier et dossiers situés dans le dossier tp Powershell mais pas les sous dossiers (Commun)

4/ Trouvez une commande pour lister les <u>fichiers et tous les sous-dossiers</u> du dossier "**TP Powershell**"

```
get-childitem -path "e:\TP Powershell" -recurse
```

5/ Trouvez une commande pour ne lister <u>que les dossiers et sous-dossiers</u> du dossier: "**TP PowerShell** "

```
get-childitem -Directory -path "e:\TP Powershell" -recurse
```

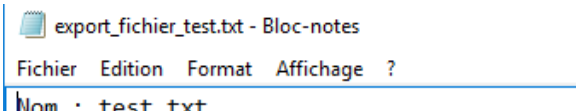
6/ Trouvez une commande pour ne lister <u>que les fichiers</u> du dossier "**TP PowerShell"** <u>mais pas de ses sous-dossiers</u>:

```
get-childitem -File -path "e:\TP Powershell"
```

7/ Trouvez une commande pour afficher <u>uniquement l'objet dossier</u> : "**TP PowerShell"**

```
Get-Item -path "E:\TP PowerShell"
```

8/ Trouvez une commande permettant d'exporter le nom, la taille en octet et la date de création du fichier "**Test.txt**" dans un fichier "**Export_fichier_test.txt"** situé dans "**Logs**". Le fichier d'export doit être conforme à l'image ci-dessous :



Aide :
Pour aller à la ligne on peut utiliser les caractères d'échappement suivants :
ALTGR+7+R : Retour charriot
ALTGR+7+N Aller à la ligne
Exemple : **Write-host " Bonjour `r`n Bob"**

```
Write-output "Nom : $((Get-Item -path "E:\TP Powershell\Travail\test.txt").Name)`r`n`
Taille : $((Get-Item -path "E:\TP Powershell\Travail\test.txt").Length) octets`r`n`
Date de création : $((Get-Item -path "E:\TP Powershell\Travail\test.txt").CreationTime)"`
 | out-file -FilePath "E:\TP Powershell\Logs\Export_fichier_test.txt"
```

9/ Trouvez une commande pour exporter le nom des fichiers "**.msc**" du dossier "**C:\Windows\System32**" dans le fichier **"E:\TP PowerShell\Travail\Fichier console msc.txt"**

```
Get-help Get-ChildItem -full #nous donne des exemples probants

Get-childitem -File -Path "C:\Windows\System32\*" -Include *msc | out-file -FilePath "E:\TP PowerShell\Travail\Fichier console msc.txt"

Get-childitem -File -Path "C:\Windows\System32\*.msc" | out-file -FilePath "E:\TP PowerShell\Travail\Fichier console msc.txt"

Get-childitem -File -Path "C:\Windows\System32" | Where-Object {$_.Extension -like "*.msc"} | out-file -FilePath "E:\TP PowerShell\Travail\Fichier console msc.txt"
```

10/ Trouvez la syntaxe pour copier les fichiers "**.msc**" du dossier "**C:\Windows\System32**" dans le dossier **"E:\TP PowerShell\Travail\Consoles"**

```
New-item -ItemType directory -Path "E:\TP PowerShell\Travail\Consoles"
Copy-Item -Path "C:\Windows\System32\*" -Include *msc -Destination "E:\TP PowerShell\Travail\Consoles"
Copy-Item -Path "C:\Windows\System32\*.msc" -Destination "E:\TP PowerShell\Travail\Consoles"
```

10/ Trouvez une commande pour compter le nombre de console ***.msc** du dossier **"E:\TP\PowerShell\Travail\Consoles" :**

Avec une propriété liée au fichier :

```
(Get-childitem -File -Path "E:\TP PowerShell\Travail\Consoles").count
```

Avec un "**pipe**" et un "**Measure-object**" :

```
Get-childitem -File -Path "E:\TP PowerShell\Travail\Consoles" | Measure-Object
```

11/ Trouvez la syntaxe pour exécuter le fichier "**lusrmgr.msc**" situé dans le dossier : **"E:\TP PowerShell\Travail\Consoles"**

```
Start-Process -FilePath "E:\TP PowerShell\Travail\Consoles\lusrmgr.msc"

Invoke-Item -Path "E:\TP PowerShell\Travail\Consoles\lusrmgr.msc"

Get-Item -Path "E:\TP PowerShell\Travail\Consoles\lusrmgr.msc" | Invoke-Item
```

STOP
PASSER PPTX
Les fournisseurs

Exercice 9.3 : les fournisseurs Powershel

Le but de cet exercice est de travailler avec certains fournisseurs vus en cours.

OBLIGATOIRE : Faites un Snapshot (Menu VM Snapshot Take a Snapshot) nommé **"Avant fournisseur"**

1/ Exécutez la commande suivante : **Get-PSDrive**

Qu'affiche cette commande ?

Les fournisseurs powershell :
Alias
FileSystem
Certificate
Environment
Function
Registry
Variable
Wsman (Fournit l'accès aux informations de configuration de WS-Management (Web Services for Management).)

Combien de providers sont gérés par Powershell ?

```
get-psdrive | select provider -Unique | Measure-Object
```

```
Affiche 8 provider
```

2/ Exécutez les commandes suivantes :

Set-Location hklm:
Get-ChildItem

Ouvrez la console "REGEDIT" et naviguez jusqu'à la branche "**HKLM**" et développez la :
Quelle difference constatez vous ente Powershell et REGEDIT ?

Aucune

Désormais, pour chaque action en powershell verifies les actions dans la console REGEDIT.

3/ Le but de cet exercice est d'ajouter, à l'aide du registre, l'icône "Bibliothèque" sur le bureau:

Pour réaliser l'opération il faut modifier la clef de registre **"{031E4825-7B94-4dc3-B131-E946B44C8DD5}"** situé dans : **"HKEY_LOCAL_MACHINE\SOFTWARE\Microsoft\Windows\CurrentVersion\Explorer\HideDesktopIcons\NewStartPanel",** qui a la valeur **1** en hexadécimal par défaut.

La commande **"Set-Itemproperty"** permet de modifier une valeur d'une clef du registre

Exécutez la commande suivante : (le "`" est réalisé avec les touches ALTGR + 7+ENTER et permet le retour à la ligne dans ISE)

Set-Itemproperty `
-path "HKLM:\SOFTWARE\Microsoft\Windows\CurrentVersion\Explorer\HideDesktopIcons\NewStartPanel" `
-Name "{031E4825-7B94-4dc3-B131-E946B44C8DD5}" -value "0"

Vérifier sur le bureau que l'icône "**Bibliothèque**" est désormais présent. (F5 pour rafraichir)

4/ Pour un gain de productivité il faut désormais, à l'aide du registre, ajouter une entrée au menu contextuel (clic droit) Nouveau " **Script Windows PowerShell**" dans l'explorateur Windows :

Il faut auparavant créer un provider pointant sur la branche HKEY_CLASS_ROOT :

Exécutez les commandes suivantes :
New-PSDrive -PSProvider registry -Root HKEY_CLASSES_ROOT -Name HKCR
Get-ChildItem HKCR:

Aide:
La commande **"New-Item"** permet de créer une clef :
New-Item –Path "HKCU:\" –Name "test-Clef"

La commande **"New-ItemProperty"** permet de créer une valeur dans une clef:
New-ItemProperty -Path "HKCU:\test-clef" -Name "Valeur_test" -Value ”" -PropertyType "String"

Faites-en sorte de répondre à cet impératif avec les données suivantes :

Chemin de la clef :	**HKEY_CLASSES_ROOT\.PS1**
Clef à créer :	**ShellNew**
Type de valeur à créer :	**Chaîne**
Nom de la valeur à créer:	**NullFile**
Donnée de la valeur:	**Script Windows PowerShell**

Notez ici votre commande:

New-PSDrive -PSProvider registry -Root HKEY_CLASSES_ROOT -Name HKCR
Set-Location HKCR:

New-Item -Path HKCR:\.PS1 -Name "ShellNew"
New-ItemProperty -Name NullFile -Path "HKCR:\.PS1\ShellNew" -PropertyType string -Value "Script Windows PowerShell"

Vérifiez dans l'explorateur Windows que vous avez bien en faisant un clic droit :

Nouveau ➡ **Script Windows PowerShell**

5/ Le but de cet exercice est de pouvoir visualiser un script PowerShell dans l'explorateur Windows sans l'ouvrir à l'aide du bouton : "Volet de visualisation".

Créez le dossier : " **E:\TP PowerShell\Scripts**"

Dans ce dossier, créez un fichier "**Script.ps1**", qui contiendra le texte suivant : "# mon premier script Powershell"

Dans l'explorateur Windows sélectionnez le script "Script.ps1" puis cliquez sur l'onglet "**AFFICHAGE**" et sur le bouton "**Volet de visualisation**"

Que voyez-vous ?

Aucun apercu n'est disponible

Créez en PowerShell la valeur suivante :

Valeur:

Chemin de la valeur:	**HKEY_CLASSES_ROOT\.PS1**
Type de valeur à créer :	**Chaîne**
Nom de la valeur à créer:	**PerceivedType**
Donnée de la valeur:	**text**

Notez ici votre commande:
```
New-ItemProperty -Name PerceivedType -Path "HKCR:\.PS1" -PropertyType string -Value "text"
```

Retournez dans l'explorateur Windows sélectionnez le script "Script.ps1" puis cliquez sur l'onglet "**AFFICHAGE**" et sur le bouton "**Volet de visualisation**"

Que voyez-vous ?

On a la présence d'un aperçu:
"# mon premier script Powershell

6/ A l'aide des variables d'environnement réalisez un script qui affichera les éléments suivants :

Nommez ce script: "**Variables_ENV.ps1**" et sauvegardez le dans: "**E:\TP PowerShell\scripts"**

Bonjour "**nom d'utilisateur courant**",
Voici quelques caractéristiques de votre ordinateur :
Nom d'ordinateur : "**nom de l'ordinateur**"
Nombre de processeur : "**nombre processeur**"
Architecture processeur : "**Architecture processeur**"
Vos fichiers temporaires se trouvent à cet endroit: "**Chemin des fichiers temp et tmp**"

Script: "Script.ps1" dans "**E:\TP PowerShell\Scripts**":

```
Write-host "Bonjour $env:UserName"
Write-host "Voici quelques caractéristiques de votre ordinateur: "
Write-host "Nom d'ordinateur : $env:COMPUTERNAME"
Write-host "Nombre de processeur : $env:NUMBER_OF_PROCESSORS"
Write-host "Architecture processeur : $env:PROCESSOR_ARCHITECTURE"
Write-host "Vos fichiers temporaires se trouvent à cet endroit:  "
$env:TEMP
$env:tmp
```

STOP
PASSER PPTX
Les variables et boucles

Exercice 9.4 : Variables et boucles

Exercice 9.4.1 : LEs Variables

1/ Affecter à la variable **$loc**, le résultat de la commande **Get-Location**.

```
$loc = Get-Location
```

Afficher les propriétés et les méthodes de la variable **$loc**……
```
$loc | gm
```

Afficher le chemin du dossier courant contenu dans cette variable……
```
$loc.Path
```

Afficher les informations sur le disque contenu par cette variable……
```
$loc.drive
```

Afficher les informations sur le 'Provider' contenu par cette variable……
```
$loc.Provider
```

2/ Affecter à la variable **$lect**, le résultat de la commande **Get-PSDrive –Name E**
```
$lect = Get-PSDrive -Name E
```

Afficher les propriétés et les méthodes de la variable $lect……
```
$lect | gm
```

A partir de la variable $lect, afficher la description du lecteur E……
```
$lect.description
```

Afficher la taille en octet du volume utilisé……
```
$lect.Used
```

Afficher la taille en octet du volume libre……
```
$lect.Free
```

3/ Affecter à la variable $fichier, le résultat de la commande Get-ChildItem "E:\TP Powershell\Travail\test.txt"
```
$fichier = Get-ChildItem "E:\TP Powershell\Travail\test.txt"
```

À partir de la variable $fichier, afficher le nom du fichier……
```
$fichier.Name
```

Afficher le nom complet du fichier (avec le chemin) `$fichier.FullName`

Afficher l'extension seule du fichier…… `$fichier.Extension`

Afficher la date du dernier accès…… `$fichier.LastAccessTime`

À l'aide d'une méthode de la variable $fichier, supprimer le fichier… `$fichier.Delete()`

Exercice 9.4.2 : Les boucles

À l'aide d'une **boucle FOR** créez 52 dossiers de **"Semaine-01"** à **"Semaine-52"** dans **"E:\TP Powershell\Archives".**
Dans chaque dossier créez un fichier nommé "archive-semainexx.txt" correspondant à la semaine (ex : "archive-semaine01.txt" pour le dossier Semaine-01)

```
For ($i=1 ; $i -le 52 ; $i++)
{
    if ($i -le 9)
    {
    New-item -ItemType Directory -Path "E:\TP Powershell\Archives\Semaine-0$i"
    New-item -ItemType File -Path "E:\TP Powershell\Archives\Semaine-0$i\archive-semaine0$i.txt"
    }
    else
    {
    New-item -ItemType Directory -Path "E:\TP Powershell\Archives\Semaine-$i"
    New-item -ItemType File -Path "E:\TP Powershell\Archives\Semaine-$i\archive-semaine$i.txt"
    }
}
```

2/ À l'aide d'une **boucle Foreach** traitez tous les dossiers et fichiers du dossier : **"E:\TP Powershell\Archives".**

Dans cette boucle Foreach insérez une boucle While qui testera la présence de chaque dossier, si le dossier existe, supprimez le.

```
$dossiers = Get-ChildItem -Path "E:\TP Powershell\Archives" -Recurse

    forEach ($i in $dossiers)
    {
        while( (test-path -Path $i.FullName) -eq $true )
        {
        Remove-item -Path $i.FullName -Recurse
        }
    }
```

2/ Faites un menu avec une **boucle Do While** qui se présentera comme suit :

```
##############################################################################
Menu d'administration de gestion d'un dossier "semaine-xx dans "E:\TP Powershell\Archives"
##############################################################################

Saisir "S" si vous voulez supprimer un dossier choisi par vos soins
Saisir "R" si vous voulez renommer un dossier choisi par vos soins
Saisir "Q" pour quitter le menu

Votre choix ?
```

Explication:
Tant que l'utilisateur ne saisira pas la touche Q ce menu s'affichera à l'écran.
Si l'utilisateur saisi la touche "S" le programme lui demandera de choisir le nom d'un dossier (semaine-xx) et le dossier sera supprimé.

Si l'utilisateur saisi la touche "R" le programme lui demandera de choisir le nom d'un dossier (semaine-xx) et le dossier sera renommé.

```
Do
{
cls
Write-host
"#########################################################################################"
Write-host " Menu d'administration de gestion d'un dossier `"Semaine-xx`" dans `"E:\TP
Powershell\Archives`" "
Write-host
"#########################################################################################"
Write-host
Write-host " Saisir `"S`" si vous voulez supprimer un dossier choisi par vos soins"
Write-host " Saisir `"R`" si vous voulez renommer un dossier choisi par vos soins"
Write-host " Saisir `"Q`" pour quitter le menu"
Write-host
Write-host
$choix = Read-host " Votre Choix ?"

    if ($choix -eq "s")
    {
    cls
    $nom_doss = Read-host "Veuillez saisir le nom d'un dosssier à supprimer (Semaine-xx)"
    Remove-Item -recurse -Path "E:\TP Powershell\Archives\$nom_doss"
    cls
    Write-host " Le dossier $nom_doss a été supprimé !`n`n`n"
    pause
    cls
    }
    if ($choix -eq "r")
    {
    cls
    $nom_doss = Read-host "Veuillez saisir le nom d'un dosssier à renommer (Semaine-xx)"
    $new_name = Read-host "Veuillez saisir le nom du nouveau dossier"
    Rename-Item -Path "E:\TP Powershell\Archives\$nom_doss" -NewName $new_name
    cls
    Write-host " Le dossier $nom_doss a été renommé !`n`n`n"
    pause
    cls
    }
}
While ($choix -ne "Q")
```

STOP
PASSER PPTX
Les tableaux

Exercice 9.5 : Les tableaux

Exercice 9.5.1 : Le piège du tableau !

1/ Exécutez la commande suivante : **Get-Process**
Qu'affiche cette commande ?

Liste de tous les processus en cours sur la machine……

2/ Exécutez les commandes suivantes :

Get-Process | Foreach { $process+= $_.Name }
$process

Quelle est le problème au niveau de $process ?

La variable $process ne contient qu'un élément avec les noms des processus, formant une unique chaine de caractère

Quelle est le type (**GetType()**) de $process ?

type string car PowerShell via le pipe a ajouté les éléments un par un donc Powershell n'a pas créé de tableau.
Pour que PowerShell créer un tableau automatiquement il faut un ajout de plusieurs éléments en une fois

3/ Exécutez les commandes suivantes :

$process = (Get-Process).Name
$process

Pourquoi le résultat maintenant s'affiche sous forme de tableau ?

PowerShell a créé un tableau automatiquement car plusieurs éléments (Process) ont été affecté à la variable $process en une fois.

Vérifier le type (**GetType()**) de $process ?

Array (tableau)

Conclusion sur le fonctionnement de PowerShell quant à l'ajout d'une ou plusieurs valeurs dans une variable.

Pour que PowerShell créer un tableau automatiquement il faut un ajout de plusieurs éléments en une fois.
Ou alors déclarer un tableau vide ($tab=@()) et dans ce cas cela fonctionne toujours.:

#Initialisation d'un tableau vide
$process=@()
Get-Process | Foreach { $process+= $_.Name }

Exercice 9.5.2 : tableaux simples

1/ Vous avez à votre disposition le tableau suivant :

$Fruits = @("Abricot","Ananas","Cerise","Fraise","Framboise","Groseille","Mirabelle","Mûre","Raisin","Tomate")

Affichez la première valeur du tableau `$Fruits[0]`

Affichez la dernière valeur du tableau `$Fruits[-1]`

Affichez les trois premières valeurs du tableau `$Fruits[0..2]`

Affichez les deux dernières valeurs du tableau `$Fruits[-2..-1] ou $Fruits[-2,-1]`

Affichez le nombre d'éléments du tableau `$Fruits.count ou $fruits.Length`

Affichez toutes les valeurs du tableau qui commencent par "m `$fruits -like "m*"`

Affichez toutes les valeurs du tableau qui finissent par "e `$fruits -like "*e*"`

Exercice 9.5.2 : tableaux associatifs

1/ Lors d'un exercice vous devez traiter un tableau afin de créer une arborescence de ressources :

Dossiers	*Fichiers*
E:\Ressources	A lire.txt
E:\ Ressources\41RT	Consignes.txt
E:\ Ressources\41RT\1Cie	1Cie.txt
E:\ Ressources\41RT\2Cie	2Cie.txt
E:\ Ressources\41RT\CCL	CCL.txt

a/ Votre Administrateur, le SGT DUGUIDON, vous donne pour vous aider une syntaxe PowerShell afin de traiter le tableau ci-dessus mais il n'a pas bien compris les cours PowerShell lorsqu'il était stagiaire à Rennes.

```
$tab = [ordered] @(
                "A lire.txt" = "E:\Ressources" ;
```

```
"Consignes.txt" ="E:\ Ressources\41RT" ;
"1Cie.txt" ="E:\ Ressources\41RT\1Cie" ;
"2Cie.txt" ="E:\ Ressources\41RT\2Cie" ;
"CCL.txt.txt" ="E:\ Ressources\41RT\CCL"
)
```

b/ Copier-coller ce code dans PowerShell ISE et exécutez-le, que se passe-t-il?

```
Au caractère Ligne:1 : 8
+ $tab = [ordered] @(
+        ~~~~~~~~~~~~
L'attribut ordered ne peut être spécifié que sur un nœud de littéral de hachage.
```

La table de hachage n'est pas correcte :

```
$tab = [ordered] @{
                   "A lire.txt" = "E:\Ressources" ;
                   "Consignes.txt" ="E:\ Ressources\41RT" ;
                   "1Cie.txt" ="E:\ Ressources\41RT\1Cie" ;
                   "2Cie.txt" ="E:\ Ressources\41RT\2Cie" ;
                    "CCL.txt.txt" ="E:\ Ressources\41RT\CCL"
                  }
```

c/ Corrigez le code du SGT DUGUIDON pour un résultat efficient.
Notez ici votre nouveau code PowerShell :

Remplacer les "(" par des "{" (voir ci-dessous le même code avec des "{,}" en rouge) :

```
$tab = [ordered] @{ "A lire.txt" = "E:\Ressources" ;
                    "Consignes.txt" ="E:\ Ressources\41RT" ;
                    "1Cie.txt" ="E:\ Ressources\41RT\1Cie" ;
                    "2Cie.txt" ="E:\ Ressources\41RT\2Cie" ;
                    "CCL.txt.txt" ="E:\ Ressources\41RT\CCL"
                  }
```

d/ Traitez le tableau $tab avec une boucle ForEach pour créer les dossiers et les fichiers du tableau.

```
ForEach ( $i in $tab.keys )
{
new-item -ItemType Directory -Path $tab[$i]
new-item -ItemType File -Name $i -Path $tab[$i]
}
```

2/ Votre Chef vous donne un tableau Excel contenant **une arborescence à créer** sous la forme :

Chemin	***Dossiers***
E:\	Exercice
E:\Exercice	Sirano
E:\Exercice\Sirano	CO
E:\Exercice\Sirano	Diranim
E:\Exercice\Sirano	Direx
E:\Exercice\Sirano\CO	Chef
E:\Exercice\Sirano\CO	Adjoint
E:\Exercice\Sirano\Diranim	Chef
E:\Exercice\Sirano\Diranim	Adjoint
E:\Exercice\Sirano\Diranim\Direx	Chef
E:\Exercice\Sirano\Diranim\Direx	Adjoint

a/ Transposez ce tableau en initialisant un tableau associatif ordonné PowerShell (**[ordered] @{xx=yy ; xx=yy}**) :

```
$tab = [ordered] @{
                "Exercice" = "E:\" ;
                "Sirano" = "E:\Exercice" ;
                "CO" = "E:\Exercice\Sirano" ;
                "Diranim" = "E:\Exercice\Sirano" ;
                "Direx" = "E:\Exercice\Sirano" ;
                "Chef-Co" = "E:\Exercice\Sirano\CO" ;
                "Adjoint-Co" = "E:\Exercice\Sirano\CO" ;
                "Chef-Diranim" = "E:\Exercice\Sirano\Diranim" ;
                "Adjoint-Diranim" = "E:\Exercice\Sirano\Diranim" ;
                "Chef-Direx" = "E:\Exercice\Sirano\Diranim\Direx" ;
                "Adjoint-Direx" = "E:\Exercice\Sirano\Diranim\Direx"
                }
```

b/ Parcourez votre tableau avec une boucle FOREACH et créez les dossiers :

```
Foreach ( $i in $tab.keys )
{
New-item -ItemType Directory -Name $i -Path $($tab[$i])
}
```

STOP
PASSER PPTX
CIM/WMI

Exercice 9.6 : CIM/WMI

1/ Exécutez la commande suivante : **Get-Command -Module CimCmdlets**

Qu'affiche cette commande ?

Affiche la liste des applets de commande CIM

2/ A l'aide de la commande suivante : **Get-CimInstance -ClassName Win32_OperatingSystem | Select ***

Afficher la version du système d'exploitation en passant par une variable **$version**

```
$info_os =Get-CimInstance -ClassName Win32_OperatingSystem | select *
$version = $info_os.version
$version
```

3/ Exporter toutes les informations sauf celles qui commencent par "D" de la commande dans un fichier CSV nommé : "**E:\TP Powershell\Export-OSInfo.csv**"

```
Get-CimInstance -ClassName Win32_OperatingSystem `
| Select-Object -Property * -ExcludeProperty d* `
| Export-Csv -Path "E:\TP Powershell\Export-OSInfo.csv" -Encoding UTF8 -Delimiter ";" -Force
```

4/ A l'aide des commandes suivantes :

Get-CimClass -Namespace root/cimv2 | where-Object { $_.CimClassName -like "mot_clef"}
Get-CimInstance -ClassName "Classe_cible" | Select-Object -Property *

Ou "**mot_clef**" est l'objet principal de la commande (disk, bios etc..) et "**Classe_cible**" la classe CIM à traiter.

Trouvez:
Le fabricant du BIOS de la machine :

```
Get-CimClass -Namespace root/cimv2 | where-Object { $_.CimClassName -like "*bios*"}
Get-CimInstance -ClassName Win32_BIOS | fl *
$bios = Get-CimInstance -ClassName Win32_BIOS
```

```
$bios.Manufacturer
```

La taille de toutes les partitions de la machine :

```
Get-CimClass -Namespace root/cimv2 | where-Object { $_.CimClassName -like "*partition*"}
Get-CimInstance -ClassName CIM_DiskPartition | fl *
Get-CimInstance -ClassName CIM_DiskPartition | foreach { write-host "$($_.size)"}
```

Le nom de toutes les cartes réseaux :

```
Get-CimClass -Namespace root/cimv2 | where-Object { $_.CimClassName -like "*network*"}
Get-CimInstance -ClassName CIM_NetworkAdapter | fl *
Get-CimInstance -ClassName CIM_NetworkAdapter | foreach { write-host "$($_.Name)"}
```

Le "**localpath**" et le "**Status**" des profils des utilisateurs :

.....

```
Get-CimClass -Namespace root/cimv2 | where-Object { $_.CimClassName -like "*profil*"}
Get-CimInstance -ClassName Win32_UserProfile | fl *
Get-CimInstance -ClassName Win32_UserProfile | foreach { write-host "$($_.localpath)";write-host
"$($_.Status)" }
```

STOP
PASSER PPTX
Scripts

Exercice 9.7 : Scripts

Exercice 9.7.1 : Création de plusieurs utilisateurs

Le but de cet exercice est de créer rapidement plusieurs utilisateurs.

A l'aide d'une boucle FOR, créez un script qui créera 5 utilisateurs, de : **User-1** à **User-5** inclus

Les utilisateurs n'auront pas de mot de passe
Les utilisateurs auront un nom complet identique à leur nom
Les utilisateurs auront une description identique à leur nom

Attention:
La création d'un utilisateur en PowerShell ne le rend membre d'aucun groupe, il faut donc rajouter une commande dans votre script pour effectuer cette action:
Pour un utilisateur standard: ADD-LocalGroupmember –Group Utilisateurs –member ….

```
For ( $var=1 ; $var-le 5 ; $var++ )
{
New-LocalUser -Name User-$var -FullName User-$var -Description User-$var -NoPassword
Add-LocalGroupMember -Group utilisateurs -Member User-$var
}
```

OU

```
For ( $var=1 ; $var-le 5 ; $var++ )
{
New-LocalUser -Name User-$var -FullName User-$var -Description User-$var -NoPassword | Add-LocalGroupMember
-Group Utilisateurs
}
```

Vérifiez la création des 5 utilisateurs et l'appartenance au groupe Utilisateurs (onglet "Membre de") dans la console **lusrmgr.msc** de tous vos utilisateurs crées.

Exercice 9.7.2 : Changement de mot de passe des utilisateurs

!!! SNAPSHOT Obligatoire avant de continuer !!!

Sous VMware, menu VM, réalisez un cliché instantané (*Snapshot*) pour pouvoir revenir ultérieurement à cet état. Nommez-le "***Avant ex 9.2***".

Le but de cet exercice est d'attribuer le mot de passe "azerty*35" aux comptes utilisateurs "User-x", créés à l'exercice précédent, à l'aide d'une boucle ForEach.

Attention: PowerShell n'accepte que les mots de passe sécurisés, saisissez dans votre script:
$mdp = ConvertTo-SecureString -asplaintext "azerty*35" -force

Consignes:
Le script devra traiter tous les utilisateurs de la base SAM.
Le script affichera un message d'accueil: "Ce script va attribuer un mot de passe aux utilisateurs "User-x"

Si le compte est un compte "User-x", lui attribuer le mot de passe **$mdp**
Le script devra afficher : " le mot de passe du compte: User-x a bien été défini ! "

Si le compte n'est pas compte "User -x", le script devra afficher:
" Le compte xx n'est pas un compte User-x, aucune action n'est requise."

Enfin, un compteur affichera le nombre de mot de passe changé à la fin du script:
" Bilan: x utilisateurs ont un nouveau mot de passe ! " en rouge

```
Write-host ""
Write-host "Ce script va attribuer un mot de passe aux utilisateurs User-x"
Write-host ""

pause

$users = Get-LocalUser

$mdp = ConvertTo-SecureString -AsPlainText azerty*35 -Force
$count = 0

ForEach ( $i in $users )
{
   if ($i.Name -like "User-*")
   {
   Set-localuser -Name $i.Name -Password $mdp
   Write-host " le mot de passe du compte: $($i.name) a bien été défini ! "
   Write-host ""
   $count++
   }
   else
   {
   Write-host " ==> Le compte $($i.Name) n'est pas un compte `"Adm-x`", aucune action n'est requise."
   Write-host ""
   }
}

Write-host ""
Write-host " Bilan: $count utilisateurs ont un nouveau mot de passe ! " -ForegroundColor Red
```

Ouvrez une session avec les 5 utilisateurs pour tester le nouveau mot de passe…

Exercice 9.7.3 : Lister et exporter les services

Le but de cet exercice est de lister les services avec une couleur différente selon leurs états (démarrés ou non) puis de les exporter vers un fichier texte.

Consignes:
Le script affichera un message d'accueil: "Ce script va lister les services avec une couleur différente selon leurs états. Un export de ces services sera disponible dans le dossier : "**E:\TP_Powershell\Logs_Services**"

Le script devra tester la présence du dossier **Logs_Services** dans **E:\ TP_Powershell** et le créer si nécessaire.
Si le dossier n'existe pas, le script devra afficher : " Le dossier **Logs_Services** n'existait pas mais il vient d'être créé."
Si le dossier existe, le script devra afficher : " Le dossier **Logs_Services** existe déjà le script peut continuer...",

Le script devra traiter tous les services de la machine.

Si le service est démarré, afficher: " le service xx est bien démarré ! " en vert
Exporter le nom du service, le nom d'affichage et son statut sous la forme: **Nom du service || Nom d'affichage || Statut** vers un fichier nommé : **Services_Running.txt**

Au début du fichier d'export le texte : "Journal des services démarrés du: GDH" doit apparaitre.

Si le service n'est pas démarré, afficher: " le service xx n'est pas démarré ! " en rouge
Exporter le nom du service, le nom d'affichage et son statut sous la forme: **Nom du service || Nom d'affichage || Statut** vers un fichier nommé : **Services_Not_Running.txt**

Au début du fichier d'export le texte : "Journal des services stoppés du: GDH" doit apparaitre.

A la fin du script, un compteur affichera le nombre de services démarrés en jaune et le nombre de services arrêtés en rouge.

```
Write-host
Write-host "Ce script va lister les services avec une couleur différente selon leurs états. Un
export de ces services sera disponible dans le dossier E:\Logs"
Write-host

if ( !(Test-Path "E:\Logs") )
{
New-Item -Itemtype Directory -Name Logs -Path E:
Write-host "Le dossier Logs dans E:\ n'existait pas mais il vient d'être créé."
}
else
{
Write-host "Le dossier Logs dans E:\ existe déjà le script peut continuer..."
}

$fic_running = "E:\Logs\Services_Running.txt"
$fic_not_running = "E:\Logs\Services_Not_Running.txt"
$srv = Get-service
$count_running = 0
$count_not_running = 0

 "" | Out-File -FilePath $fic_running -append
 "#####################################################"  >> $fic_running
 "Journal des services démarrés du $(Get-Date)"  >> $fic_running
 "#####################################################"  >> $fic_running
 ""  >> $fic_running

 "" >> $fic_not_running
 "#####################################################" >> $fic_not_running
 "Journal des services stoppés du $(Get-Date)" >> $fic_not_running
 "#####################################################" >> $fic_not_running
 "" | Out-File -FilePath $fic_not_running -append

    ForEach ( $i in $srv )
```

```
    {
        if ($i.Status -eq "Running" )
        {
        Write-host " le service : $($i.Name) est bien démarré ! " -ForegroundColor Green
        "$($i.ServiceName) || $($i.DisplayName) || $($i.status)" >> $fic_running
        Write-host
        $count_running++
        }
        else
        {
        Write-host " le service : $($i.Name) n'est pas démarré ! " -ForegroundColor Red
        "$($i.ServiceName) || $($i.DisplayName) || $($i.status)" >> $fic_not_running
        Write-host
        $count_not_running++
        }
    }

Write-host
Write-host "#######################################"
Write-host " Bilan des services démarrés et arrétés:"
Write-host "#######################################"
Write-host

Write-host " $count_running Services sont démarrés " -ForegroundColor yellow
Write-host
Write-host " $count_not_running Services ne sont pas démarrés " -ForegroundColor Red
```

Exercice 9.7.4 : Lister et exporter des informations sur les utilisateurs

Le but de cet exercice est de lister les utilisateurs selon leurs états d'activation et d'exporter certaines informations vers un fichier texte.

Consignes:
Le script affichera un message d'accueil: "Ce script va lister les utilisateurs selon leurs états d'activation. Un export de ces informations se trouvera dans le dossier : **"E:\TP_Powershell\Logs_Users"**

Le script devra tester la présence du dossier **Logs_Users** dans **E:\TP_Powershell** et le créer si nécessaire.
Si le dossier n'existe pas, le script devra afficher : " Le dossier Logs_Users n'existait pas mais il vient d'être créé."
Si le dossier existe, le script devra afficher : " Le dossier Logs_Users existe déjà le script peut continuer...",

Le script devra traiter tous les utilisateurs de la base SAM.

Si l'utilisateur est activé, afficher : " le compte : xx est activé " **en jaune**
Si l'utilisateur activé a eu un mot de passe modifié, afficher : " le mot de passe du compte xx a été changé le: xx "
Si l'utilisateur activé n'a pas eu un mot de passe modifié, afficher: " le mot de passe du compte xx n'a pas été changé "

Exporter le nom de l'utilisateur, le nom complet et son statut sous la forme: **Nom de l'utilisateur || Nom complet || Description || Last connexion || SID** vers un fichier nommé : **Users_enabled.txt**

Au début du fichier d'export le texte : "Journal des comptes activés du GDH" doit apparaitre.

Si l'utilisateur est désactivé, afficher : " le compte : xx est désactivé " **en rouge**
Si l'utilisateur désactivé a eu un mot de passe modifié, afficher : " le mot de passe du compte xx a été changé le: xx "
Si l'utilisateur désactivé n'a pas eu un mot de passe modifié, afficher: " le mot de passe du compte xx n'a pas été changé "

Exporter le nom de l'utilisateur, le nom complet et son statut sous la forme: **Nom de l'utilisateur || Nom complet || Description || Last connexion || SID** vers un fichier nommé : **Users_disabled.txt**

Au début du fichier d'export le texte : "Journal des comptes désactivés du GDH" doit apparaitre.

A la fin du script, un compteur affichera le nombre d"utilisateur activés en jaune et le nombre d"utilisateur désactivé en rouge.

```
Write-host
```

```
Write-host " Ce script va lister les utilisateurs selon leurs états d'activation. Un export de
ces informations se trouvera dans le dossier E:\TP_Powershell\Logs_Users"
Write-host ""

if ( !(Test-Path "E:\TP_Powershell\Logs_Users") )
{
New-Item -Itemtype Directory -Name "Logs_Users" -Path "E:\TP_Powershell"
Write-host "Le dossier Logs_Users n'existait pas mais il vient d'être créé."
}
else
{
Write-host "Le dossier Logs_Users existe déjà, le script peut continuer..."
}

$fic_enabled = "E:\TP_Powershell\Logs_Users\Users-enabled.txt"
$fic_disabled = "E:\TP_Powershell\Logs_Users\Users-disabled.txt"
$users = Get-localUser
$count_enabled = 0
$count_disabled = 0

"" >> $fic_enabled
"#####################################################" >> $fic_enabled
"Journal des utilisateurs activés du $(Get-Date)"  >> $fic_enabled
"#####################################################" >> $fic_enabled
"" >> $fic_enabled

"" >> $$fic_disabled
"#####################################################" >> $fic_disabled
"Journal des utilisateurs désactivés du $(Get-Date)" >> $fic_disabled
"#####################################################" >> $fic_disabled
"" >> $fic_disabled

    ForEach ( $i in $users )
    {
        if ($i.enabled -eq $true )
        {
        $count_enabled++
        Write-host " le compte : $($i.Name) est activé" -ForegroundColor Yellow

         "nom: $($i.Name) || Nom complet: $($i.FullName) || Description: $($i.Description) ||
Last Connexion: $($i.Lastlogon) || SID: $($i.SID)"  >> $fic_enabled

            if ($i.PasswordLastSet -ne $null)
            {
            Write-host " le mot de passe du compte $($i.Name) a été changé le:
$(($i.PasswordLastSet).day)/$(($i.PasswordLastSet).month)/$(($i.PasswordLastSet).year) "
            }
            else
            {
            Write-host " le mot de passe du compte $($i.Name) n'a pas été changé"
            }
        Write-host

        }
        if ($i.enabled -eq $false )
        {
        $count_disabled++
        Write-host " le compte : $($i.Name) est désactivé" -ForegroundColor Red

         "nom: $($i.Name) || Nom complet: $($i.FullName) || Description: $($i.Description) ||
Last Connexion: $($i.Lastlogon) || SID: $($i.SID)"  >> $fic_disabled
         ""  >> $fic_enabled

            if ($i.PasswordLastSet -ne $null)
```

```
            {
            Write-host " le mot de passe du compte $($i.Name) a été changé le:
$($i.PasswordLastSet)"
            }
            else
            {
            Write-host " le mot de passe du compte $($i.Name) n'a pas été changé"
            }
        Write-host
        }
    }

Write-host
Write-host "##############################################"
Write-host " Bilan des utilisateurs activés et désactivés:"
Write-host "##############################################"
Write-host
Write-host " $count_enabled utilisateurs sont activés " -ForegroundColor yellow
Write-host
Write-host " $count_disabled utilisateurs sont désactivés " -ForegroundColor Red
```

Exercice 9.7.5 : Faire un menu de gestion des utilisateurs

Le but de cet exercice est de réaliser un menu avec la commande SWTICH qui proposera plusieurs options afin de manager les utilisateurs.

Consignes:
Le menu devra se présenter comme suit:

```
"================================================================================="
"========================= Menu de gestion des utilisateurs ========================="
"================================================================================="
"              1: Saisissez '1' pour afficher les utilisateurs"
"              2: Saisissez '2' pour afficher les groupes"
"              3: Saisissez '3' pour sauvegarder le profil d'un utilisateur"
"              4: Saisissez '4' pour sauvegarder le profil de tous les utilisateurs"
"              5: Saisissez '5' pour supprimer toutes les sauvegardes"
"              6: Saisissez '6' pour quitter"
"================================================================================="
```

Aide:

- Au début de votre script saisissez la commande suivante pour supprimer à l'affichage certaines erreurs parasites:
  $ErrorActionPreference = "SilentlyContinue"
- Si vous devez afficher la liste des utilisateurs dans le script saisissez:
  Get-localUser | ft
- Pour la copie n'oubliez pas le **-recurse** et le **-Force**

But:
Quel que soit le choix de l'utilisateur et tant que l'utilisateur n'a pas saisi la touche "6" pour quitter, ce menu doit toujours être affiché, ceci afin de réaliser au besoin plusieurs opérations. (Do..While)

Détail de votre futur script:
Si l'utilisateur saisi la touche "1":
Pour chaque utilisateur de la base SAM le script devra afficher:
Le nom (Nom: toto)
Le nom complet (Nom complet: Toto Bob)
La description (Description: Le compte à TOTO)
Le SID (SID: xxxxx)

Si l'utilisateur saisi la touche "2":

Pour chaque groupe de la base SAM le script devra afficher:
Le nom
La description
Le SID

Si l'utilisateur saisi la touche "3":

Le script devra afficher la phrase suivante: " Sélection d'un utilisateur pour une sauvegarde vers **E:\Backup_Users**..."
Le script devra afficher la liste des utilisateurs
Le script affichera une demande à l'utilisateur pour choisir un utilisateur parmi la liste

Si l'utilisateur existe:

Si l'utilisateur possède un dossier à son nom dans **c:\Users\nom de l'utilisateur**
Création du dossier : "**E:\Backup_users**"
Copie du dossier **c:\Users\nom de l'utilisateur** dans "**E:\Backup_users**"
Le script devra afficher la phrase suivante: "La sauvegarde s'est bien déroulé..."

Si l'utilisateur ne possède pas de dossier à son nom dans **c:\Users\nom de l'utilisateur**
Le script devra afficher la phrase suivante: " L'utilisateur ne s'est pas connecté, il n'a pas de profil !"
Le script devra afficher la phrase suivante: " Aucune sauvegarde n'a été effectuée !"

Si l'utilisateur n'existe pas:

Le script devra afficher la phrase suivante: " l'utilisateur xx n'existe pas " en rouge
Le script devra afficher la phrase suivante: "La sauvegarde s'est bien déroulé..."

Si l'utilisateur saisi la touche "4":

Le script devra afficher la liste des utilisateurs
Le script devra afficher la phrase suivante: "Sauvegarde de tous les utilisateurs"
Si le dossier **"E:\Backup_users\All_Users"** n'existe pas le créer

Pour chaque utilisateur

Si l'utilisateur possède un dossier à son nom dans **C:\Users\nom de l'utilisateur**
Copier son dossier vers le dossier **"E:\Backup_users\All_Users"**
Le script devra afficher la phrase suivante: "La sauvegarde de l'utilisateur xx s'est bien déroulé..."

Si l'utilisateur ne possède pas de dossier à son nom dans **C:\Users\nom de l'utilisateur**
Le script devra afficher la phrase suivante: " l'utilisateur xx n'a pas de profil. Aucune sauvegarde n'est nécessaire..."

Si l'utilisateur saisi la touche "5":

Le script devra afficher la phrase suivante: " Suppression de toutes les sauvegardes..."

Tant que l'utilisateur n'appuiera pas sur la touche "o", le script devra afficher:
!!! ATTENTION !!! Si vous confirmer l'action suivante, le dossier E:\Backup_users et son contenu seront supprimés (cette action est irréversible...) " en rouge
Ainsi que la phrase : " Etes-vous sûr de vouloir continuer ? (o/n) "

Si l'utilisateur saisi la touche "o" :
Le script devra afficher la phrase suivante: "Suppression en cours..."
Suppression du dossier "**E:\Backup_users**"
Le script devra afficher la phrase suivante: "Le dossier **E:\Backup_users** et son contenu ont bien été supprimé."

Si l'utilisateur saisi autre chose que les touches de 1 à 6:

Le script devra afficher la phrase suivante: " Vous n'avez pas sélectionné la bonne valeur..."

```
if ( !(test-path "e:\Backup_users") )
{
New-Item -ItemType Directory -Name "Backup_users" -Path "E:\"
$backup_users = (Get-item -Path "E:\Backup_users").FullName
}
else
{
$backup_users = (Get-item -Path "E:\Backup_users").FullName
```

```
}

cls
write-host ""
do
{
$ErrorActionPreference = "SilentlyContinue"
Write-Host
"==================================================================================="
Write-Host "========================= Menu de gestion des utilisateurs
========================="
Write-Host
"==================================================================================="
Write-Host "              1: Saisissez '1' pour afficher les utilisateurs"
Write-Host "              2: Saisissez '2' pour afficher les groupes"
Write-Host "              3: Saisissez '3' pour sauvegarder le profil d'un utilisateur"
Write-Host "              4: Saisissez '4' pour sauvegarder le profil de tous les utilisateurs"
Write-Host "              5: Saisissez '5' pour supprimer toutes les sauvegardes"
Write-Host "              6: Saisissez '6' pour quitter"
Write-Host
"==================================================================================="

Write-Host ""
$choix = Read-Host " Veuillez sélectionner un choix du menu (1 .. 6), SVP "
Write-Host ""

    Switch ($choix)
    {
        "1"
         {
         $users = Get-localuser
         Write-host ""

             ForEach ( $i in $users )
             {
             Write-host " Nom: $($i.Name) "
             Write-host " Nom complet: $($i.FullName) "
             Write-host " Description: $($i.Description) "
             Write-host " SID: $($i.SID) "
             Write-Host ""
             }
         }

        "2"
         {
         $groups = Get-localgroup
         Write-host ""

             ForEach ( $i in $groups )
             {
             Write-host " Nom: $($i.Name) "
             Write-host " Description: $($i.Description) "
             Write-host " SID: $($i.SID) "
             Write-Host ""
             }
         }

        "3"
         {
         cls
         Write-Host " Sélection d'un utilisateur pour une sauvegarde vers E:\Backup_Users..."
         Write-Host ""
         Get-localuser | ft
         Write-Host ""
```

```
        $choix_user = read-host "Veuillez sélectionner un utilisateur (==> Name)"

            if ( !(Get-LocalUser -Name $choix_user) )
            {
            Write-Host ""
            Write-Host ""
            Write-Host " l'utilisateur $choix_user n'existe pas "  -ForegroundColor Red
            }
            else
            {

               if ( (test-path "C:\Users\$choix_user") -eq $true )
               {
               New-Item -ItemType Directory -Name $choix_user -Path $backup_users
               Copy-Item -Path "C:\Users\$choix_user" -recurse -Destination $backup_users
-Force
               Write-Host ""
               Write-Host "La sauvegarde s'est bien déroulé..."
               }
               else
               {
               Write-Host ""
               Write-Host " L'utilisateur ne s'est pas connecté, il n'a pas de profil !"
               Write-Host " Aucune sauvegarde n'a été effectuée !"
               }
            }
        }

        "4"
        {
        cls
        Get-localuser | ft
        $users = Get-localuser
        Write-Host ""
        Write-Host "Sauvegarde de tous les utilisateurs"
        Write-Host ""
        pause

            if ( (test-path "E:\Backup_users\All_Users") -eq $false )
            {
            New-Item -ItemType Directory -Name All_Users -Path $backup_users
            }

            Foreach ( $i in $users )
            {
                if ( (test-path "C:\Users\$($i.Name)") -eq $true )
                {
                Copy-Item -Path "C:\Users\$($i.Name)" -recurse -Destination
"$backup_users\All_Users" -Force
                Write-Host ""
                Write-Host "La sauvegarde de l'utilisateur $($i.NAme) s'est bien déroulé..."
                }
                else
                {
                Write-Host ""
                Write-Host "l'utilisateur $($i.NAme) n'a pas de profil. Aucune sauvegarde n'est
nécessaire... " -ForegroundColor Red
                }
            }
        }

        "5"
        {
        cls
```

```
            do
            {
            Write-Host ""
            Write-Host " Suppression de toutes les sauvegardes..."
            Write-Host ""
            Write-Host ""
            Write-Host " !!! ATTENTION !!! Si vous confirmer l'action suivante, le dossier
E:\Backup_users et son contenu seront supprimés (cette action est irreversible...) "
-ForegroundColor Red
            Write-Host ""
            Write-Host ""
            Write-Host ""
            $choix_supp = Read-Host " Etes-vous sûr de vouloir continuer ? (o/N) "
            Write-Host ""
                if ( $choix_supp -eq "o" )
                {
                cls
                Write-Host "Suppression en cours..."
                Write-Host ""
                Write-Host ""
                Remove-item -Recurse -path $backup_users -Force
                Write-Host "Le dossier $backup_users et son contenu ont bien été supprimé."
                Write-Host ""
                Write-Host ""
                pause
                }
            cls
            }
            While ($choix_supp -ne "o")
        }

        "6"
        {
        cls
        Write-Host
        Write-Host  "Vou savez choisi de quitter le menu ..."
        Write-Host
        Write-Host
        pause
        exit
        }

        Default
        {
        Write-host " Vous n'avez pas sélectionné la bonne valeur..."
        }
    }

Write-Host ""
Write-Host ""
pause
cls
}
while ($choix -ne '6')
```

Annexe 1 : Commandes pour obtenir de l'aide

Afficher de l'aide sur une commande : Get-Help *Commande* *(ex : Get-Help Get-ChildItem ou help Get-ChildItem)*
Afficher les exemples : Get-Help *Commande* –Examples
Afficher l'aide complète : Get-Help *Commande* –Full

Afficher les alias : Get-Alias

Afficher la liste des méthodes et des propriétés des objets : *Commande* | Get-member

Annexe 2 : Commandes pour gérer les fichiers et les dossiers

Se déplacer dans les dossiers : Set-Location *chemin* *(ex : Set-Location c:\temps)*

Afficher le chemin du dossier courant : Get-Location

Afficher le contenu d'un dossier : Get-ChildItem

Créer un dossier : New-Item "E:\Travail" -ItemType directory

Créer un fichier avec du texte New-Item "E:*nomFichier.txt*" -ItemType file –Value "*texte*"

Supprimer un fichier ou un dossier : Remove-Item "E:*nomFichier.txt*"

Déplacer un fichier : Move-Item "E:*nomFichier.txt*" -Destination "F:*nomFichier.txt*"

Déplacer un dossier : Move-Item "E:\Travail" -Destination "F:\"

Renommer un fichier ou dossier : Rename-Item "E:*nomFichier.txt*" -NewName "E:\F*ichier.txt*"

Copier un fichier : Copy-Item "E:*nomFichier.txt*". -Destination "F:\"

Copier un dossier avec ses fichiers :Copy-Item "E:\Travail" -Destination "F:\" –Recurse

Tester l'existence d'un fichier ou dossier : Test-Path "E:*nomFichier.txt*"

Annexe 3 : Variables, propriétés et méthodes des objets

Le nom d'une variable commence toujours par $, il peut inclure tout caractère alphanumérique ou le trait de soulignement.

Windows PowerShell permet de créer des variables qui sont pour l'essentiel des objets nommés. La sortie de toute commande Windows PowerShell valide peut être stockée dans une variable.

Exemple : $loc = Get-Location

Il est possible d'utiliser Get-Member pour afficher des informations sur le contenu de variables.

Exemple : $loc | Get-Member (idem Get-Location | Get-Member)

Pour un tableau :
Get-Member –InputObject $tab (membres d'un tableau)
$tab | Get-Member (membres des objets contenues dans un tableau)

Le nom de la variable suivi du point permet d'accéder aux propriétés de l'objet référencé par la variable, exemple pour la propriété Path de la variable $loc.
Exemple : $loc.Path

Remarque : l'usage de la touche tabulation [tab] permet de compléter le nom de la propriété.

De même, l'exécution d'une méthode (action) d'un objet :
Exemple : $fichier.Delete()

Remarque : Pour les méthodes, ne pas oublier les parenthèses avec ou sans paramètre.